

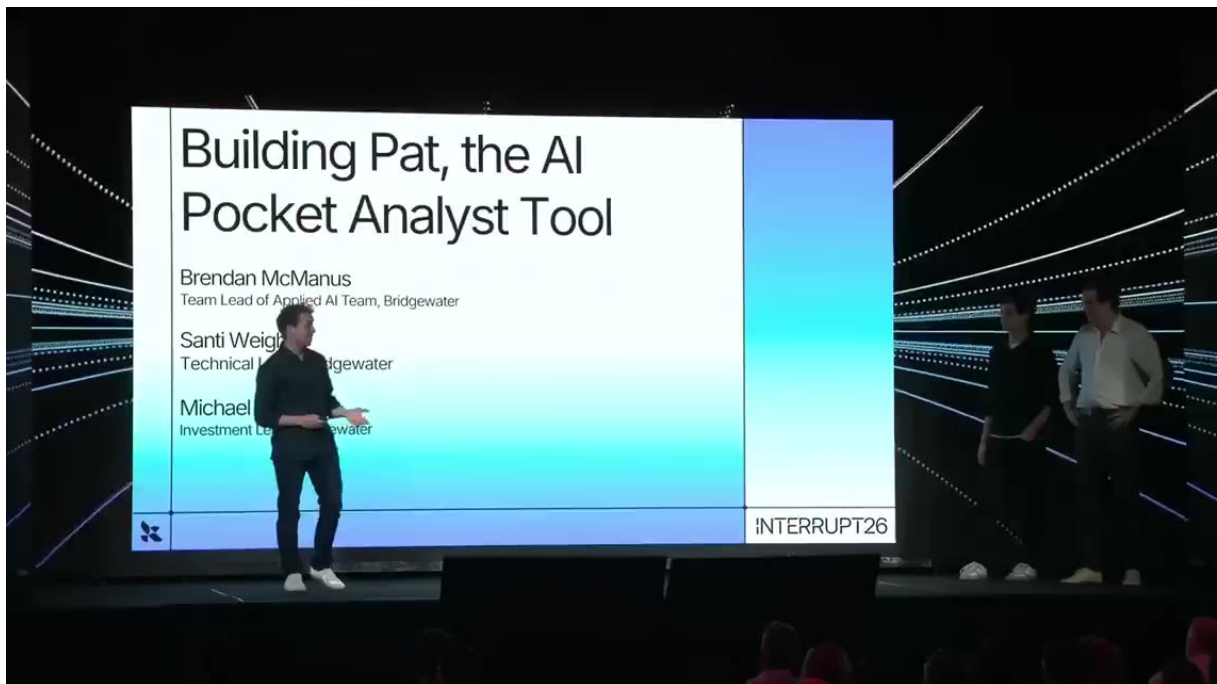
Building PAT, the AI Pocket Analyst Tool

Bridgewater Associates — Applied AI Team
INTERRUPT26 Conference

Brendan McManus — Team Lead, Applied AI Team
Michael Ran — Investor Lead
Santi Weight — Technical Lead

Full session transcript with original slides
Compiled 2026-08-05

Part 1 — Brendan McManus (Team Lead, Applied AI Team)



Title slide — Building Pat, the AI Pocket Analyst Tool (INTERRUPT26)

Hello everyone. My name is Brendan McManus, the team lead of the applied AI team at Bridgewater Associates, a systematic macro hedge fund. I've been at Bridgewater for almost a decade now, where I started as a software engineer, then became a systematic investor and researcher, and for the past few years, I've been focused heavily on bridging the gap between investing and technology.

I'm here today with my colleagues, Michael Ran and Santi Weight, our investor lead and technical lead of this project respectively, to talk to you about a great tool that we built internally called Pat, the Pocket Analyst. Now, by the end of this talk, all of you should have seen how we've built an AI analyst that's capable of performing hours of expert research work in minutes, that's been internally deployed to hundreds of investors, and that learns from every single interaction. On top of that, we'll also show you exactly how we architected such a thing.

So, before we actually show you what we built, I wanted to go through some quick framing on Bridgewater's approach to AI. Bridgewater has been spending decades, 50 years, thinking about how to encode markets and economies into compounding systems. And all of this really started with what you're seeing on the screen: our 1980 bond system written down on a yellow legal pad. The idea here was quite simple. Every time you wanted to make a trade, write down exactly the rules for why you think that trade makes sense. Write down the exact causal logic, because once you do this, another investor can take a look at what you've written, help you to figure out where you went wrong, and help you to improve upon this process. Every time you learn something new, you scratch out a rule, you write down a new one, and this creates a pretty incredible learning process that has been the foundation of everything that we've done at Bridgewater for the past 50 years.

Bridgewater's AI Journey

Systemization

1980 Bond System

Expert System

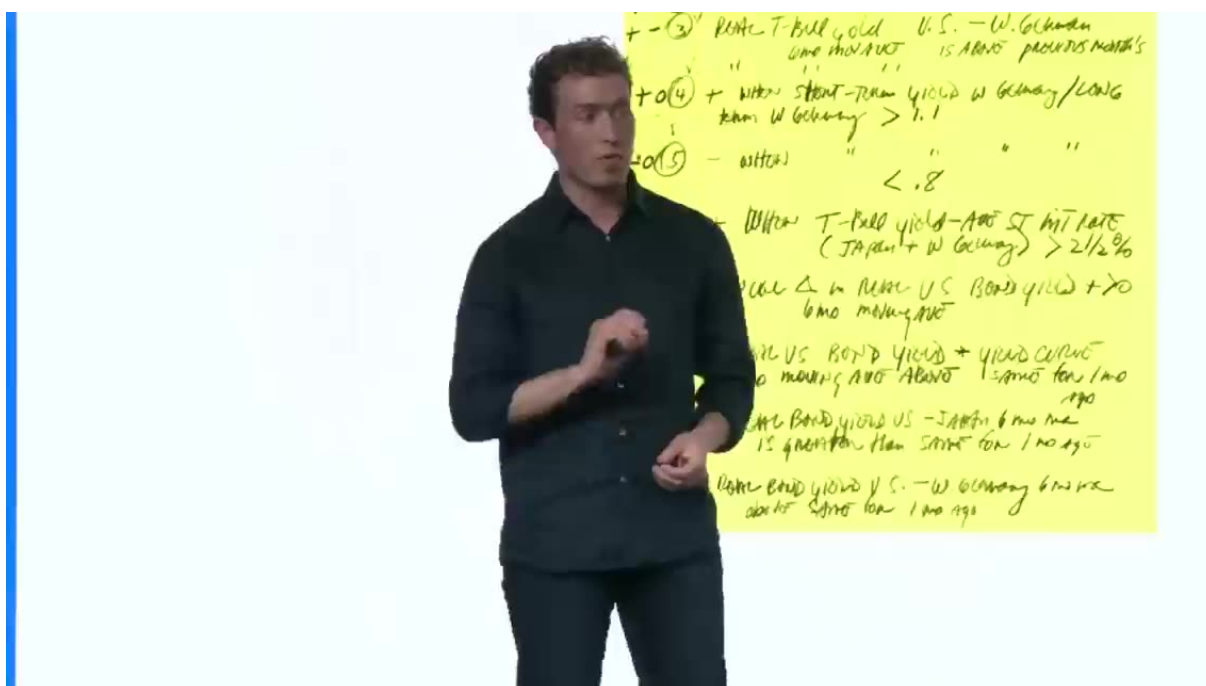
Our Process Today

INTERRUPT26

for distribution purposes only. Please remove the "Copyright Disclaimers and Other Information" located at the end of this presentation.

BRIDGEWATER

Bridgewater's AI Journey — Systemization (1980 Bond System) to Expert System

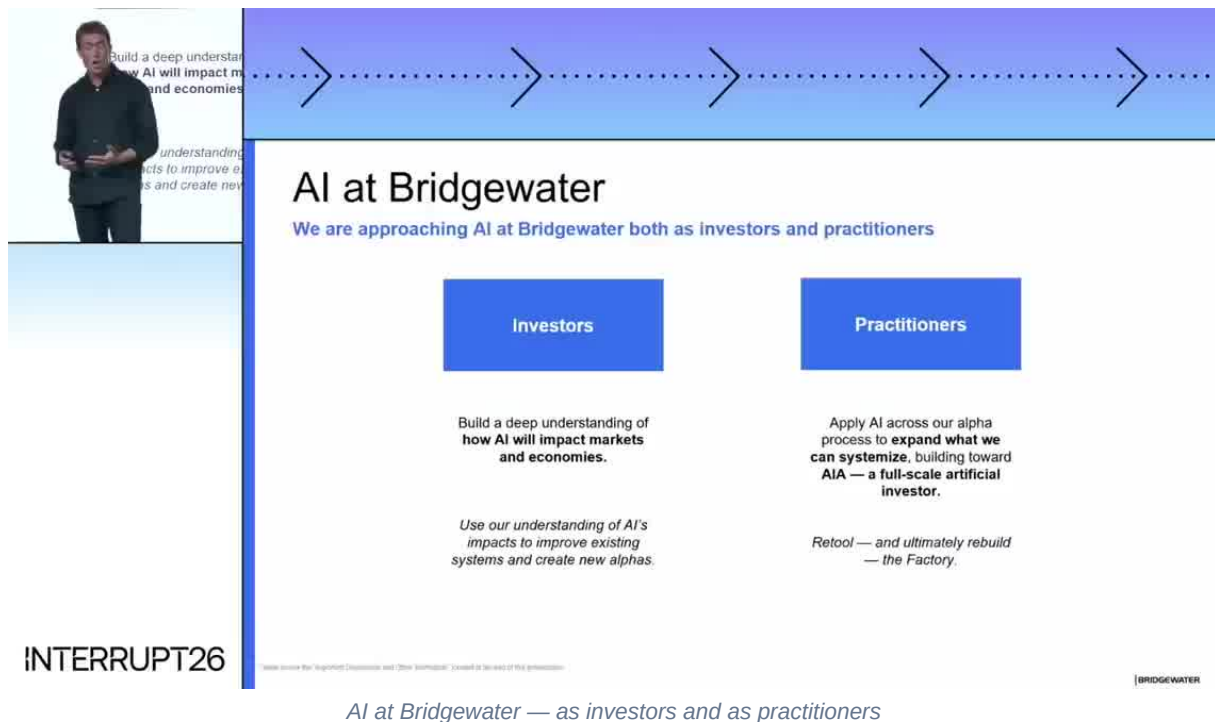


The 1980 bond system, handwritten on a yellow legal pad

Over the decades, we've compounded upon this process pretty significantly as we've taken every lesson, every methodology, every rule that we've developed about the trades that we make and how and why, and we've codified them into an expert system that's both machine-readable and human-readable. And it's really now that we're just sitting on a pretty tremendous trove of data. And all of this data is really what sets us up well for the AI era. We didn't have to go back and write down everything for agents; it was already there for us to draw upon.

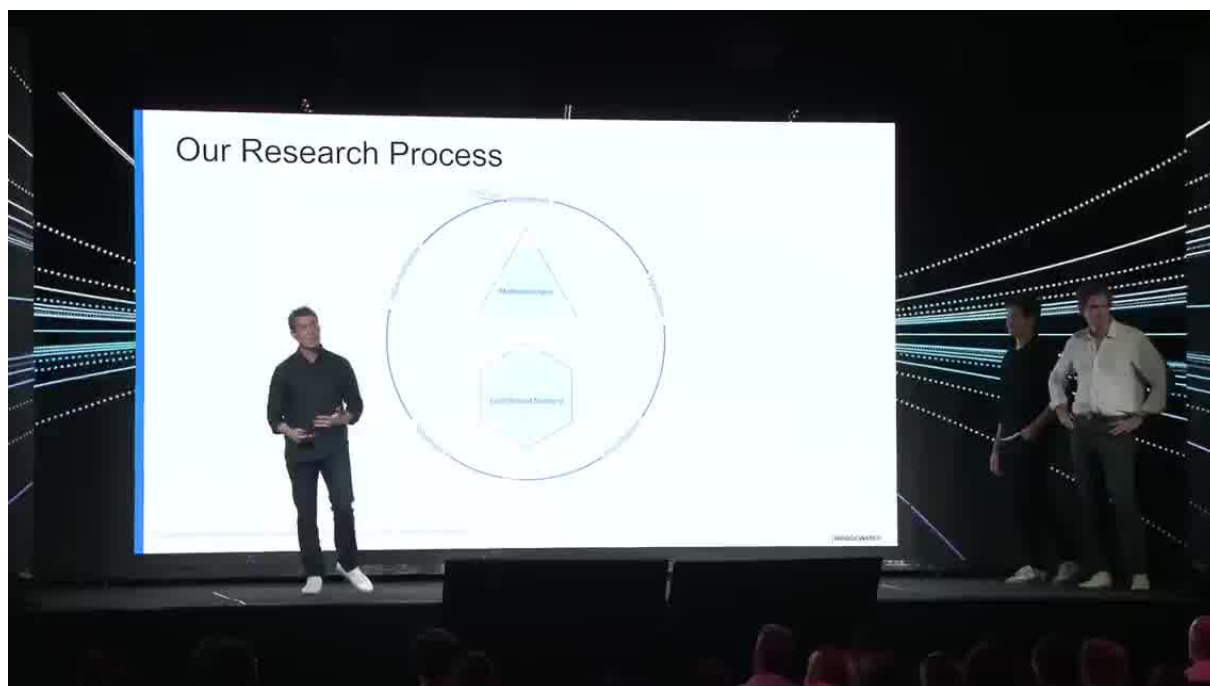
And so, before we get into the tool a bit more, I wanted to talk about how we're approaching AI at Bridgewater more generally. We're doing this in two ways. The first is as investors. As investors, we

need to deeply understand every major dynamic that's shaping global markets and economies. Just as we had to understand COVID or the recent oil supply shock, we've also had to understand AI. What's the shape of the supply-demand mismatch? What's driving the build-out, and how do these things ultimately impact markets? This understanding is table stakes for us as investors.



The second way that we're approaching AI is as practitioners, which is what we're mostly going to be spending our time on here today. As practitioners, we're applying AI across all aspects of our research process, with the ultimate goal of building out an artificial investor that's capable of the full range of activities that our human investors are performing every single day.

So, what do our human investors actually do? Well, we think about it as a research circle. Investors are constantly perceiving what's happening in the outside world, formulating questions about what is true and what they might potentially be missing, performing analytical investigations to try to answer these questions, synthesizing their findings, and ultimately taking anything that they learned and putting it back into the compounded understanding — the expert systems that we built out. It's really this last step here that's key. Everything that we learn as a function of this process gets put back into a shared memory that humans are able to draw upon.



Our Research Process — perception, hypotheses, investigation, synthesis, shared memory

And so, you might even imagine that if you're going to build out an artificial investor, they need to be able to do all of the different steps of this research process. You could imagine having built out discrete sub-agents that are focused on each portion of the research circle. And that's exactly how we're approaching AI. We're building out discrete sub-agents for each of these different things that human investors must do, drawing upon the same understanding that we've built up over the last 50 years.

But today, we're only going to be talking about one of these agents that we've built out, which is really focused on the investigation portion of the research process. The kinds of deep analytical work that take our human analysts days and weeks to perform. We call this tool that we built Pat, the Pocket Analyst Tool. And just to set expectations, Pat is not about how we trade. It's really about performing deep exploratory research, enabling our investors to go after questions that they never would have had the bandwidth to go after before.

So, what did we actually build? We built this tool called Pat, the Pocket Analyst Tool, and from day one, the spec was simple: we had to enable Pat to be able to do all of the things that our human investors are doing as they're performing investigatory or analytical work. And that really starts with data. Pat had to be able to search across and read all of the different data that we have internally, from structured time-series data (for example, stock prices going back decades) to unstructured data (for example, broker-dealer research pieces that we subscribe to, or internal memos that we produce). Pat has to be able to search across and read all of it.

1

SEARCH ALL DATA

Structured time-series, unstructured research, and live positions — nothing left out.

2

USE ALL TOOLS

Every instrument our human analysts rely on is available to the Pocket Analyst.

3

DIAGNOSABLE

Pocket Analyst can explain every calculation it makes.

4

KNOWS OUR CONTEXT

Pocket Analyst understands the context of every investment idea, from the company's business model to the macro environment.

5

CONTINUAL LEARNING

Pocket Analyst learns from every interaction, improving its performance over time.

INTERRUPT26

BRIDGEWATER

Pat's five pillars — 1: Search All Data

Pat also has to be able to use all of the different tools that our human analysts have access to — all of the proprietary tools that we built out for visualization, for diagnostics, or for evaluating the quality of the indicator ideas that we built out. In addition to this, and I think this is where things might get a little bit more interesting for a technical audience, many of the analyses that Pat is running would take a human analyst many hours at a time to run. And this means that the analyses are quite complex, which means that Pat's analyses have to be completely diagnosable, not just for humans, but also for agents running in the background, reading through the traces, understanding and making sure that every calculation is correct.

2

USE ALL TOOLS

Every instrument our human analysts rely on is available to the Pocket Analyst.

3

DIAGNOSABLE

Pocket Analyst can explain every calculation it makes.

4

KNOWS OUR CONTEXT

Pocket Analyst understands the context of every investment idea, from the company's business model to the macro environment.

5

CONTINUAL LEARNING

Pocket Analyst learns from every interaction, improving its performance over time.

INTERRUPT26

BRIDGEWATER

Pat's five pillars — 2: Use All Tools

On top of this, and I think this is really where building upon all of the things that we've written down over the past 50 years starts to pay off, is that Pat knows all of our context. It has access to all of our investment processes, our frameworks. It knows exactly how our analysts are supposed to be doing work because we've been writing it down for the past 50 years. And lastly, Pat has to learn. It has to compound upon its own learning, not just for one investor, but for every investor at the firm.

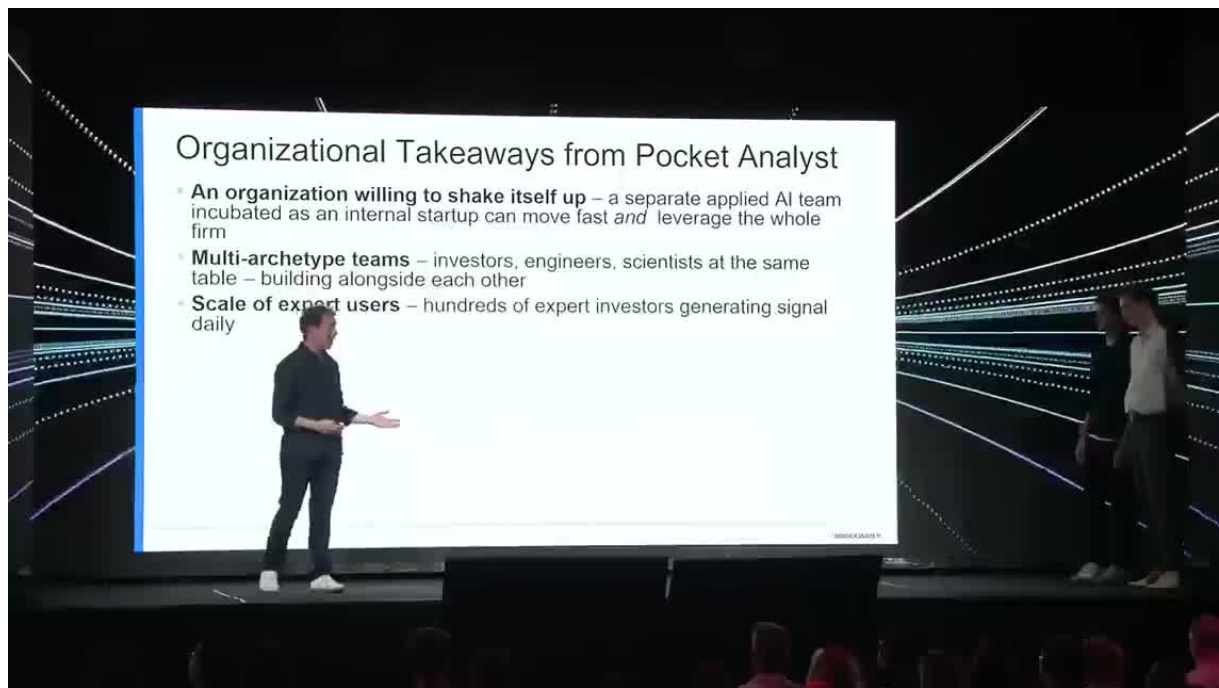
Pat's not a prototype today; it's actually been deployed internally several months ago and we now have hundreds of investors using it every single day. This is leading to a pretty incredible flywheel of improvement where, as investors are using Pat for real research, we have agents continuously running in the background, scanning through these interactions, figuring out where Pat went wrong, developing human-audited benchmarks, which then results in changes to context and also to the harnesses that we've built for Pat, which results in Pat improving not just for one, but for everyone.



The flywheel — investor uses Pat, agent extracts lesson, human adds context, Pat improves

And so lastly, before we get into what we built, I wanted to leave you with one question that I pretty commonly get, which is: how does a 50-year-old hedge fund actually build out something like what you're about to see? Well, it really starts with being willing to shake yourself up. The team that built out Pat was ultimately an internal applied AI startup that was incubated within the broader firm — the ability to move incredibly flexibly and quickly while being able to draw upon the resources of the broader firm.

In addition to this, we've established multi-archetype teams where we have investors, technologists, and scientists ultimately sitting side-by-side, building alongside each other. And I think this is really a key thing that you need if you're going to build out a product like this. Investors bring the context and the domain-specific expertise, technologists bring the architectural capability, and scientists bring the rigor. It's really these types of multi-archetype teams that are necessary if you're going to be building AI systems for expert users.



Organizational takeaways — shake yourself up, multi-archetype teams, scale of expert users

And speaking of expert users, we have hundreds of them internally using all of our AI tools, not just Pat which you're about to see, generating signal daily on how these things should be evolving and improving over time. And lastly, we have a pretty incredible compounding ecosystem to plug into. 50 years of shared data, tooling, methodologies, all built out not just for our human analysts to plug into, but also for agents on our journey to build out a fully artificial investor that is capable of everything that humans are able to do today. And so with that, I'm going to pass things off to Michael Ran, Investor Lead, to go through a demo of what we built and take you through the product architecture.

Part 2 — Michael Ran (Investor Lead)

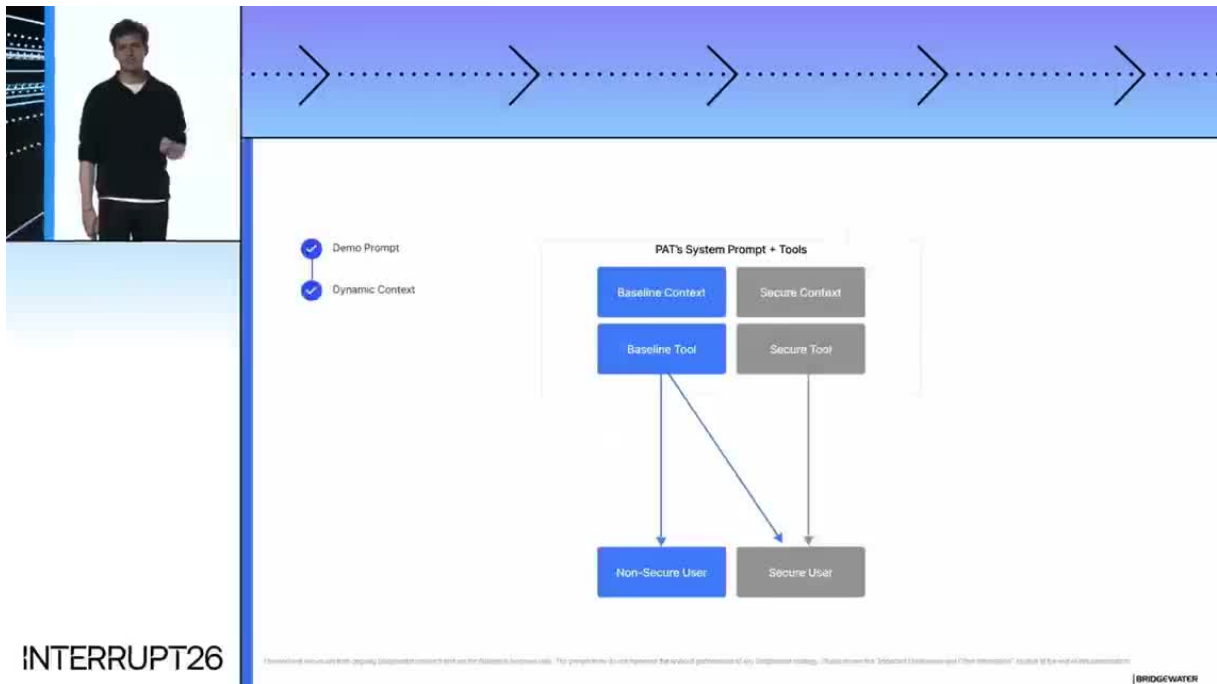


Section — Demo and Product Architecture

All right, thank you Brendan. My name is Michael Ran, I'm the investor lead working on the Pocket Analyst. And a little bit about myself, I've been here at Bridgewater for five years where I joined as a technologist, but then have spent most of my time in an investor role. And then nowadays, the thing I'm mostly focusing on is how we can infuse AI into our investment process. So with that being said, I'd love to jump into a demo of the Pocket Analyst to highlight its capabilities and show you all how investors at Bridgewater are using it.

So behind me, you can see the Pat landing page which has the prompt we'll be using for today's demo. Where the gist of it is, we're asking Pat to look at how markets have responded to the recent conflict in the Middle East and to compare today's events to similar historical episodes. Then lastly, we ask Pat to put together some visualizations that highlight how similar and/or dissimilar today is relative to previous oil supply shocks. This is a real question we've been working on at Bridgewater over the past couple of months, and Pat has been a tool leveraged by investors to help expedite our research process.

But before we submit the prompt, I'd like to spend a moment talking about an interesting security problem that we ran into when designing Pat's harness. So our starting point was, for Pat to truly be helpful, it needs to be able to access all of the information that investors have access to when conducting research. But the thing is, at Bridgewater, different investors have access to different information. So for example, you may have one investor who has access to what our positions are across all markets today, and it's important for that person's Pat to also have this information. But then you might have analysts who are not exposed to this, and it's similarly critical that we do not accidentally leak the secure IP to those analysts.



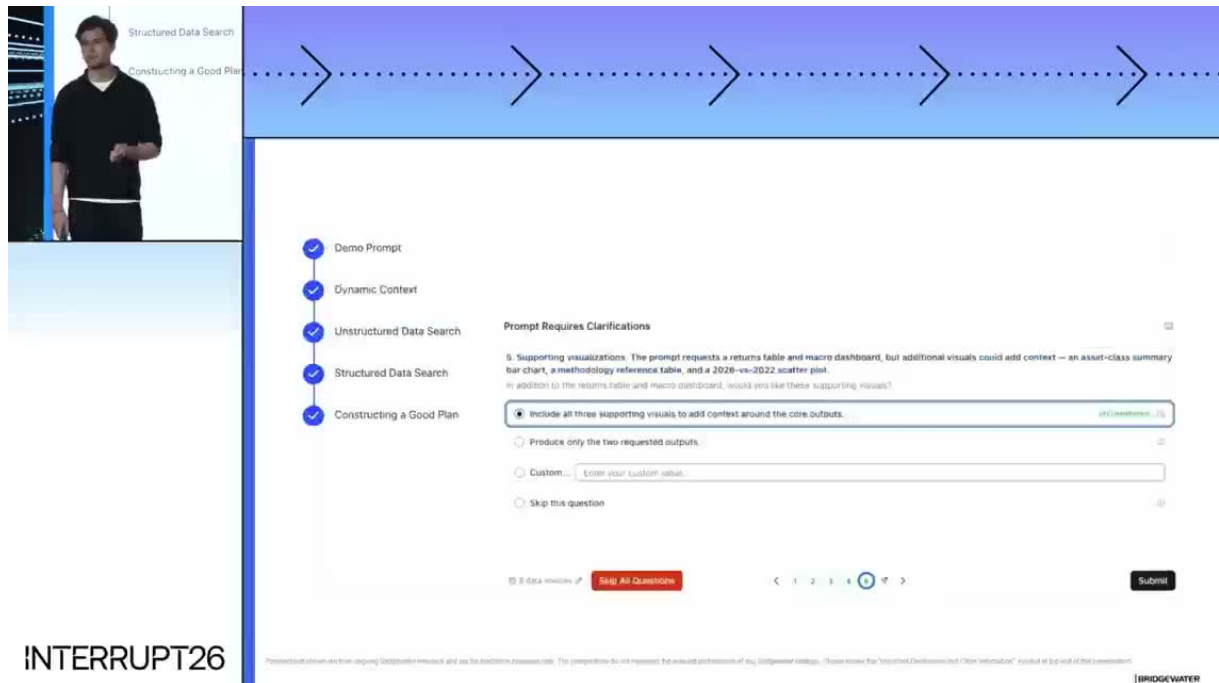
So unlike a harness like Claude Code where everyone is interfacing with the same system prompt and the same tools, each person at Bridgewater has a unique version of Pat tailored for what they can and cannot see. Now practically speaking, this uniqueness is just a function of what context and what tools each person's Pat has. So with that, we'll go ahead and submit the prompt.

And the first thing that takes place in this analysis is Pat searches the web and our vault of unstructured data to better understand what's happening in the world today and to contextualize it against history. Now web search itself is table stakes when it comes to these modern chat applications, but the real differentiator here is the breadth of unstructured content that we subscribe to that Pat is able to search over. We have a database containing millions of documents from all around the world that includes content such as broker pieces, earnings transcripts, internal emails, and more. And this database is updating in near real-time with thousands of new pieces coming in each day. And this ties back to the point I was making on the previous slide, which is for Pat to really provide leverage, it has to access everything that its user has access to, such that it can try and emulate what its users actually do.

Now once this context is gathered, the next thing that Pat does is it searches our time series database for the data required for the analysis. This is a database that contains tens of millions of series that we've been modeling internally for 50 years. And this database contains data from the outside world like the price of oil, and also internally derived concepts such as what we think inflation will be 12 months out.

Now the search agent uses kind of your traditional search techniques like RAG, reranking, etc. But the thing we found to be a huge difference maker was layering on an element of human-like inspection. And what I mean by that is when a human researcher is looking for data, they don't just anchor to what the name of the time series is. They'll do things like look at the frequency, the currency of the series, and then most importantly, whether the values in the series align with their priors. So embedding this sort of reasoning into our search agent is actually something that got us up from roughly like 50% accuracy all the way to 90.

Now after Pat has the context and data at hand, it comes back to the user with clarifying questions and potentially ideas for additional angles to explore. During Pat development, we came to the view that the plan really is the analysis. If we can create a high-quality detailed plan, we felt confident that we could consistently take this plan, intelligently execute it, and produce our desired outputs. And Santi will be diving into this a bit more later.



INTERRUPT26

Demo — Pat asks clarifying questions before locking the plan

So while agents asking questions is pretty standard in chatbots nowadays, the thing we really focused on was the substance of those questions. We devoted lots of time and energy into developing the context and benchmarks that help shape this capability. We taught Pat what makes a good research question versus a bad one. So this back and forth helps humans, who tend to under-invest in planning, flesh out what we consider to be a high-quality research plan.

Now after the ambiguous points are all resolved, Pat enters its planning phase. Where during the planning phase, the things it's doing are: A) coming up with all of the data frames that will be produced in the analysis, coming up with the schemas of all those data frames, and then most importantly, coming up with how all those data frames connect. Now this planning phase is relatively expensive from a time perspective in our analysis, but it's a cost that we pay deliberately because of what it lets us do during execution.

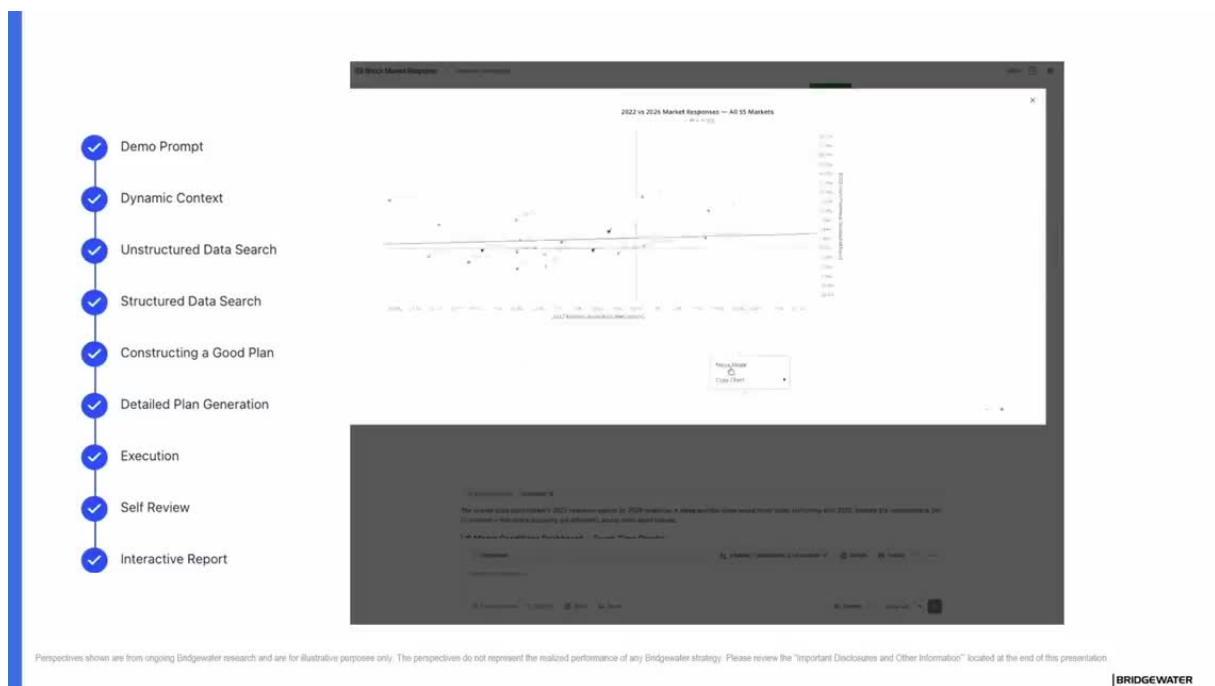
So now we have the plan locked in, and the first step of plan execution is generating the code. And given we have such a detailed plan, the thing we're able to do is generate the code for each data frame in our analysis in parallel using sub-agents. And this works because each sub-agent knows what data frames it depends on, it knows the schemas of those data frames, and it also knows what the schema of the data frame it's producing should be. So this lets it be such that you can have an analysis with say three data frames or a meaty one with something like 30 data frames, and it takes us roughly the same amount of time to generate the code in both of those scenarios.

Once we finish generating the code, we just execute the Python functions with an agent overseeing this execution and stepping in if it sees things like runtime errors or nonsensical values. And before

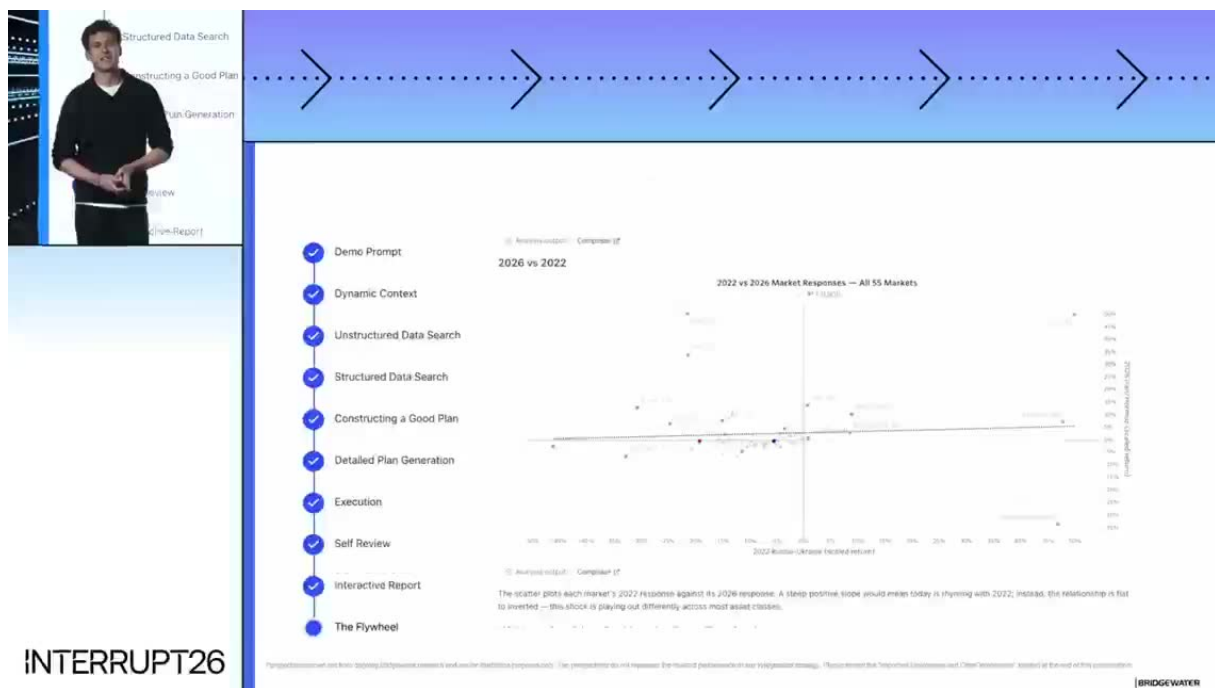
we proceed from execution, I want to call out the fact that the time series outputs from these analyses land in the same database that we actually pulled our inputs from. And I think this is important for a couple of reasons. One, it shows that any output from a Pat analysis is indistinguishable from any of the human-uploaded series that we've been producing for many years. And then more importantly, any output from a Pat analysis can serve as an input to a subsequent one. So what this does is it creates an environment where humans and agents can very easily compound and leverage each other's work.

Now after execution completes, just like you'd want your junior analyst to double-check their work before coming back to you, we want Pat to do the same. So at this point in the analysis, Pat will look at the data it's computed and the visualizations it's produced and make sure that the numbers seem sensible and the charts look clean. If it sees something that looks a bit off, Pat will take a step back, diagnose the issue, then refine the analysis and make sure it's satisfied with its result before coming back to the user.

And then the result itself is this interactive report where in the report, the visualizations look just like the ones investors at Bridgewater are producing. Pat is using the same in-house charting library and leveraging the same existing tech stack that we've been developing internally for decades. So you can see here how you can zoom in, zoom out of these charts, and you can even send the data from this interactive report to our internal charting tool where you can make further modifications on the fly.



Interactive report — 2022 vs. 2026 market responses, all 55 markets

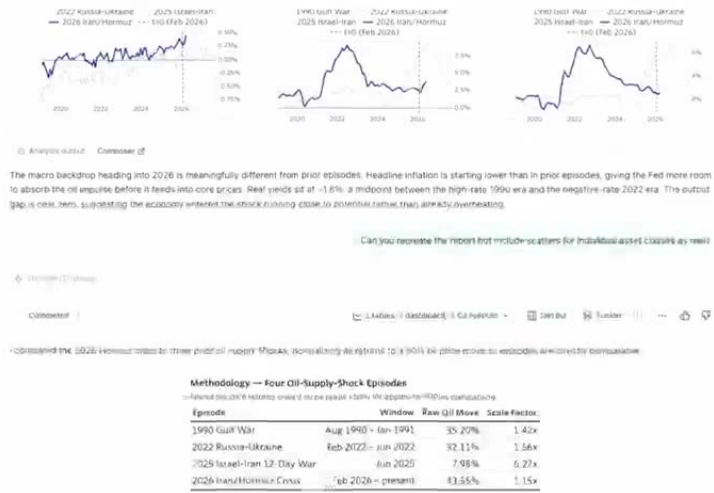


Interactive report — 2026 vs. 2022 scatter with analysis caption

And now before handing it over to Santi, I want to speak a bit to how Pat gets better as you use it. There are two primary ways in which this happens. The first is the autonomous one that Brendan alluded to earlier, where we just have agents reviewing completed conversations, looking for ways in which Pat can get smarter. And then the second one, and the one we'll be showing here, is the more explicit option where a user can kick off this learning process within the context of an analysis if they think there is something to be learned from their interaction with Pat.

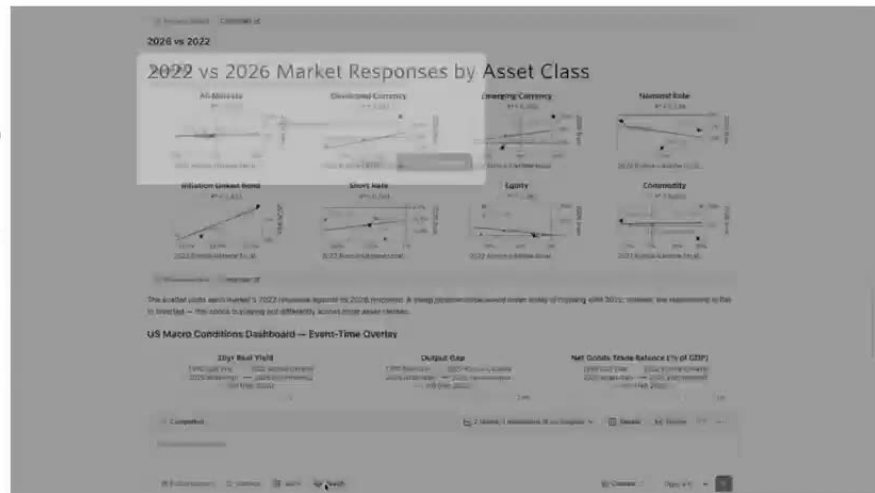
So right here, we just have the user asking for a different set of visualizations. Note the user didn't say anything was wrong, they just wanted a different perspective that they thought was important to answering the question at hand. If they think that Pat should have produced or suggested this set of visualizations from the jump, they can click the teach button, which will then spawn an agent to go through the conversation, looking for things like behavioral mistakes, context gaps, or user steering that can be front-run. The user is then able to modify this or send it as is.

- ✓ Demo Prompt
- ✓ Dynamic Context
- ✓ Unstructured Data Search
- ✓ Structured Data Search
- ✓ Constructing a Good Plan
- ✓ Detailed Plan Generation
- ✓ Execution
- ✓ Self Review
- ✓ Interactive Report
- The Flywheel



Follow-up — oil-supply-shock episode comparison (1990 Gulf War, 2022 Russia-Ukraine, 2025 Israel-Iran, 2026 Hormuz)

- ✓ Demo Prompt
- ✓ Dynamic Context
- ✓ Unstructured Data Search
- ✓ Structured Data Search
- ✓ Constructing a Good Plan
- ✓ Detailed Plan Generation
- ✓ Execution
- ✓ Self Review
- ✓ Interactive Report
- The Flywheel



Perspectives shown are from ongoing Bridgewater research and are for illustrative purposes only. The perspectives do not represent the realized performance of any Bridgewater strategy. Please review the "Important Disclosures and Other Information" located at the end of this presentation.

BRIDGEWATER

Follow-up visualizations — responses by asset class + US macro conditions dashboard

INTERRUPT26

Teach PAT

BRIDGEWATER

Teach PAT — the explicit learning entry point

And when they submit this, what happens is first, an agent on the backend will create a benchmark that we expect to fail. This shows that we can reproduce this poor behavior. It will then iterate on our context repositories or the harness itself until that benchmark passes. And then after confirming that by making this benchmark pass, we didn't cause the rest of our suite to fail, we get a Slack message with a pull request including the changes the agent wants to make to Pat. And the thing this does is that the next time a human comes to Pat with a similar question, we expect them to get the better version of Pat right out of the box. So now I'd like to hand it over to Santi for the technical overview.

Part 3 — Santi Weight (Technical Lead)

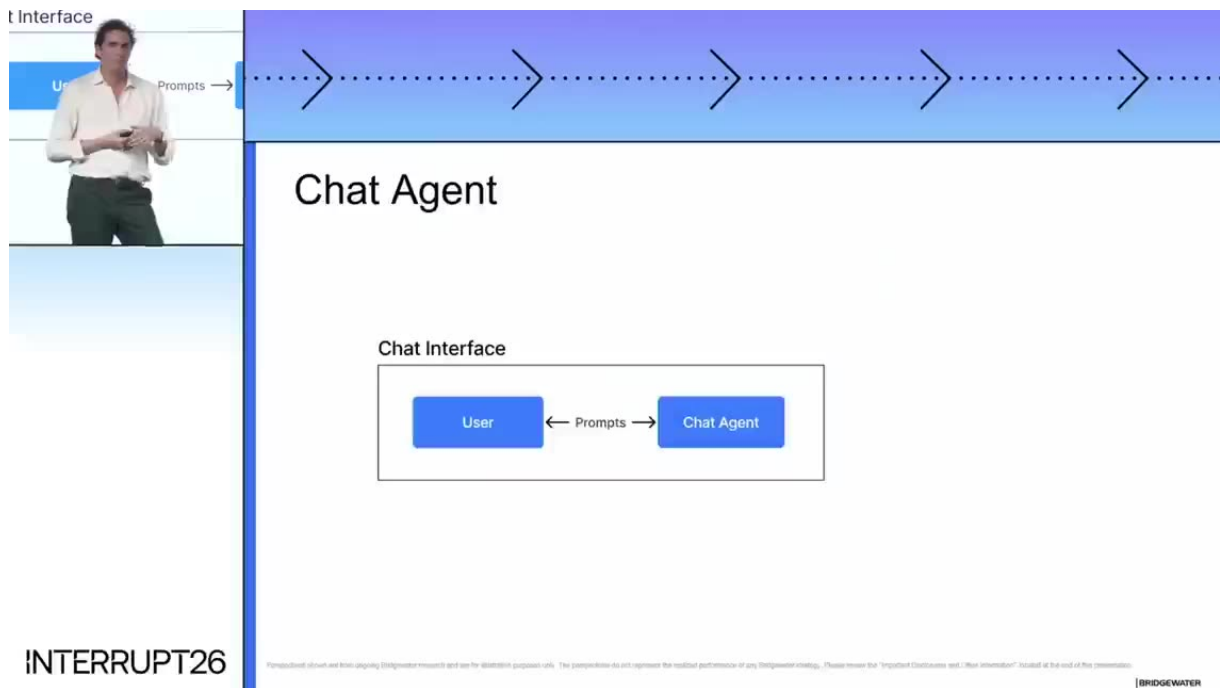


Section — Technical Overview

Hello guys. Thanks Michael. I'm Santi, I'm the tech lead for the Pocket Analyst. I'm sure many of you in the crowd are building coding agent products just like we are. And it's really hard to build one. Coding agents are really fickle, unpredictable, they often make mistakes, and then when you're really unlucky they go crazy and try and nuke your data and all the rest of it. And so it's already hard to build a good product that people like, but then it's really hard to build a product that they want to embed into their daily workflows. At a hedge fund where we're trying to trade billions of dollars, and so we can't have just vibes code be the underpinning of how these analyses go.

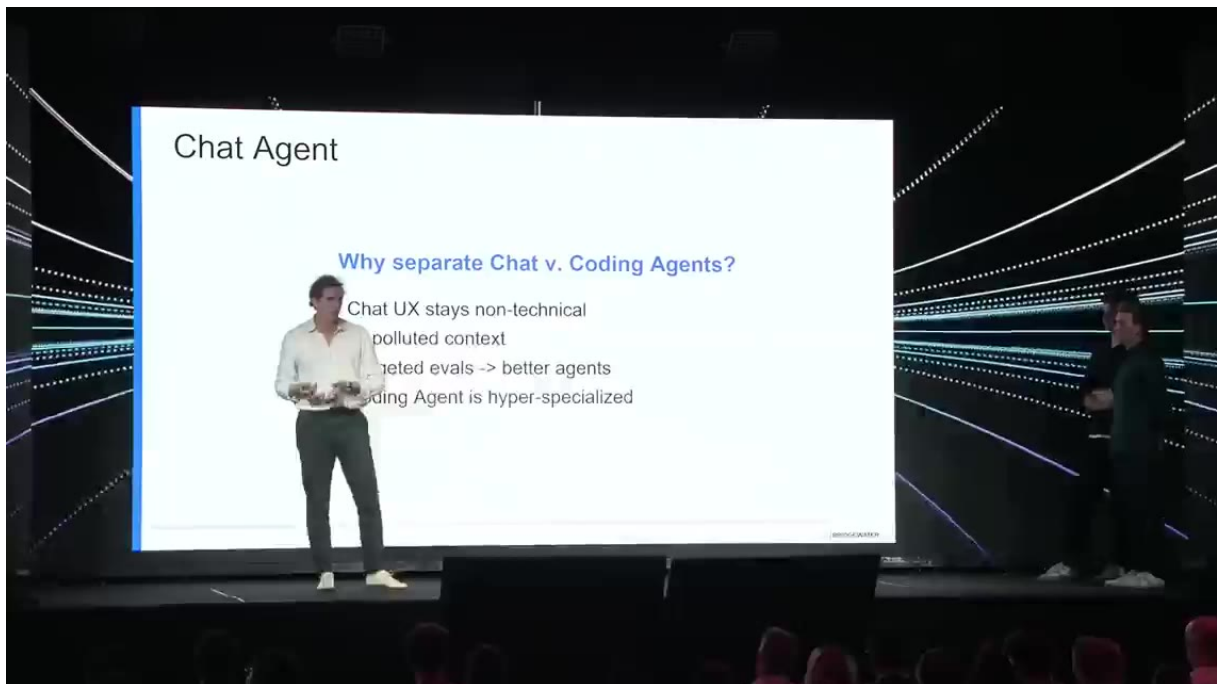
My background is in compiler theory and programming language design. And compilers have a very similar footprint of requirements, which is: they're fully deterministic, fully correct, and reliable. You can't have an off-by-one when you're flying a plane, for example. And there's also a similar shape here. A compiler takes user code and compiles it down to something like JS. And then a coding agent takes a user prompt or plan and compiles it down to Python. We really like this approach. And so I don't have a lot of time today, we're going to focus on this as a learning that maybe you guys might be able to take home to your own work.

We're going to start with the chat though. So the goal here is for the chat agent and the user to come to a common understanding of what they're trying to accomplish here. The chat agent is implemented in LangGraph. We primarily use it for persistent support, it has out-of-the-box cancellations and continuations, and we used to manage that stuff ourselves with a much worse effect. The chat agent has access to tool calls. Michael called some of these out. So there's Data Series Search, Unstructured Search, and each of these would be a talk unto themselves. And then once the chat agent knows all the data that it needs for its analysis, it's going to make a plan and invoke a tool call to a sub-agent which is our coding agent. And that's going to produce a Python pandas analysis.



Chat Agent — chat interface, prompts, tool calls

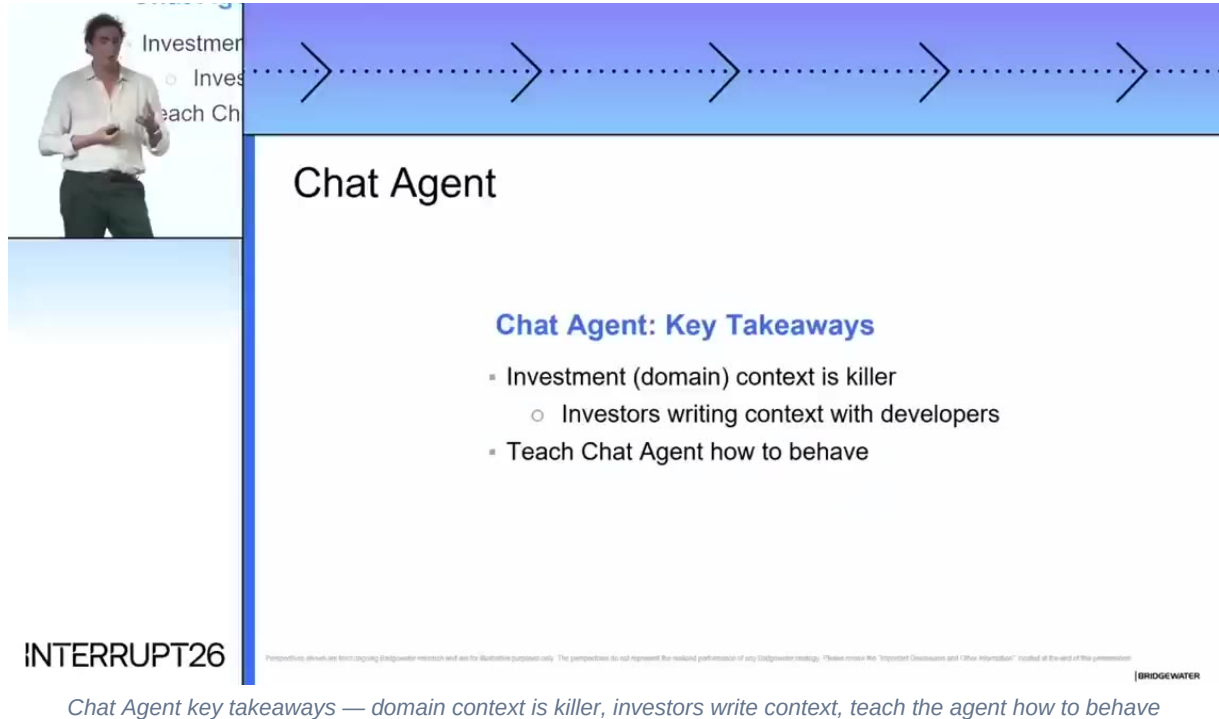
So why would you separate your two agents? Early on, we decided that our investors are not programmers by trade, and they care about investment. And so we decided to keep the chat purely about investment content. The result is that we have a product where coding is a pure implementation detail. From the chat, you can't tell that there's code under the hood. And other happy accidents are that you get unpolluted context. So each agent becomes specialized at its job and naturally improves. And we get to tailor the chat experience a lot.



Why separate Chat vs. Coding Agents — non-technical UX, unpolluted context, targeted evals, hyper-specialization

And that's what we're going to talk about in a sec right now. Is our investment domain context is really high quality. So we teach the chat agent how to talk like a Bridgewater investor. There's a lot

of jargon to teach it. And so the user and the agent both talk to each other like they're co-workers at Bridgewater. We also let our investors contribute, like we were showing that, contribute like a developer on the code base. And this is a pretty good point. Like, your user is better at writing context than you are, most likely. And so having low ego about it and letting them contribute directly is a good way to win.



The slide features a speaker on the left and a blue header with five right-pointing chevrons. The main title is 'Chat Agent'. Below it, the section 'Chat Agent: Key Takeaways' is followed by a bulleted list. At the bottom left is the text 'INTERRUPT26' and at the bottom right is the 'BRIDGEWATER' logo.

Chat Agent

Chat Agent: Key Takeaways

- Investment (domain) context is killer
 - Investors writing context with developers
- Teach Chat Agent how to behave

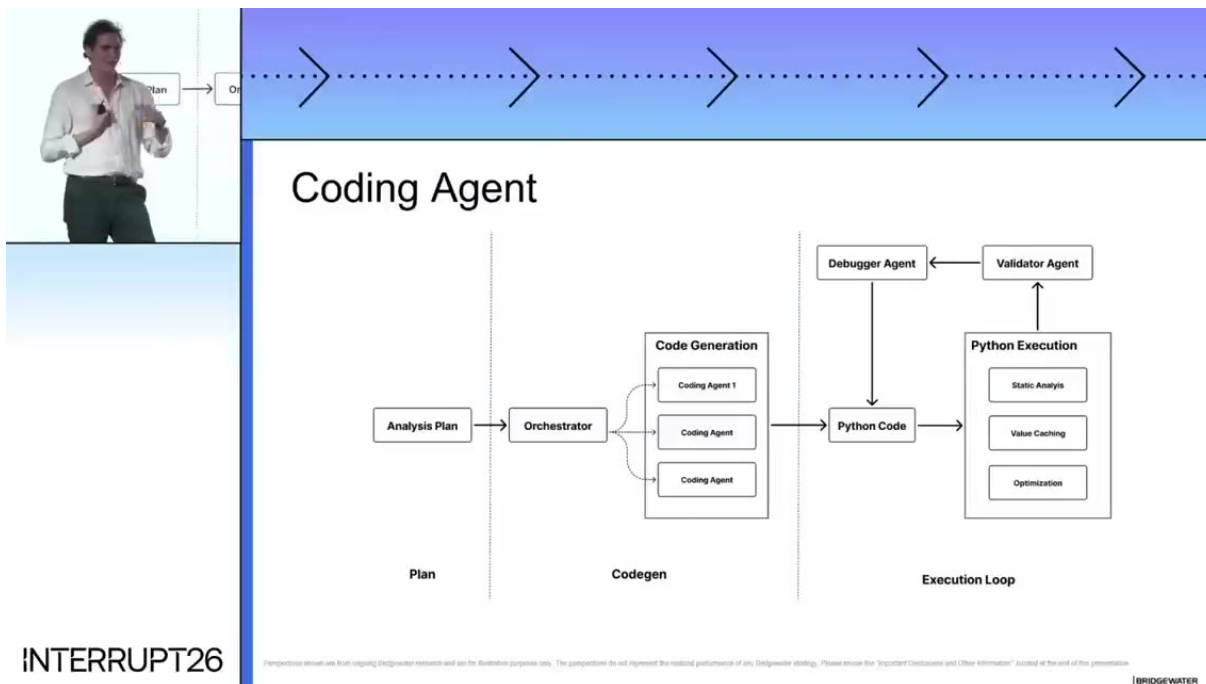
INTERRUPT26

BRIDGEWATER

Chat Agent key takeaways — domain context is killer, investors write context, teach the agent how to behave

The last point I'll make with the chat is we teach the chat agent not just context, which you end up with this nebulous context that is very high information but doesn't really feel like a workflow. And so instead, we have step-by-step guides for the agent on how to handle certain types of analyses, and it feels much more like a product at that point. It's like dependable workflows.

So the rest of the talk is about my personal favorite part, which is the coding agent. This is a high-level architecture diagram of the coding agent. Everything you see here is actually just Python code. It's implemented with LangGraph but there's no agentic orchestration. Everyone taking photos, that's great. We're going to start with the left, which is the analysis plan that the chat agent makes.

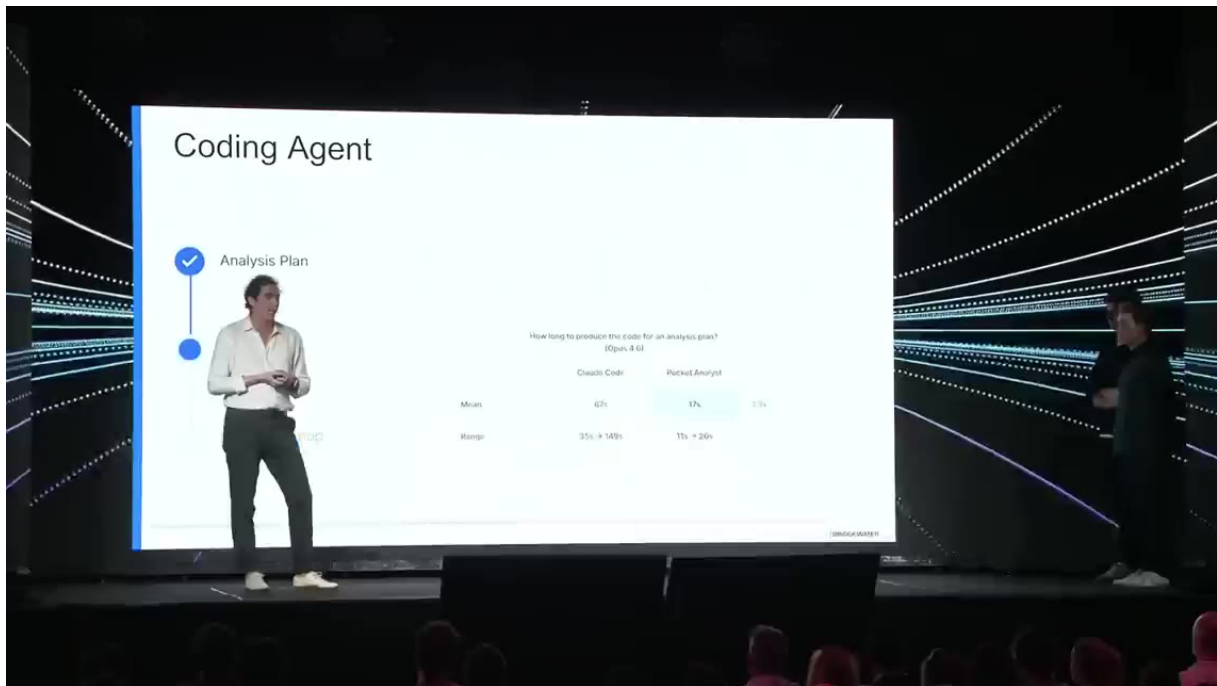


Coding Agent architecture — plan, orchestrator, parallel codegen, execution harness, validator and debugger agents

So the analysis plan is broken up into tasks. Each task maps approximately to one Python function that's going to calculate a data frame. And here's an example schema. So it gets a name, it has a description of what to calculate, and then structural and semantic information about the data frame that should come out. We expect every task to deterministically compile via LLM to a piece of code. So two LLMs operating on the same task should produce code that when run is semantically equivalent with the same output values. Exactly the same.

The result is that our analysis plan isn't just like a to-do list like you might have in Claude Code. Instead, we think of it as a natural language Python project. And because we have so much detail, now we're going to go into codegen, and we can apply some fancy techniques.

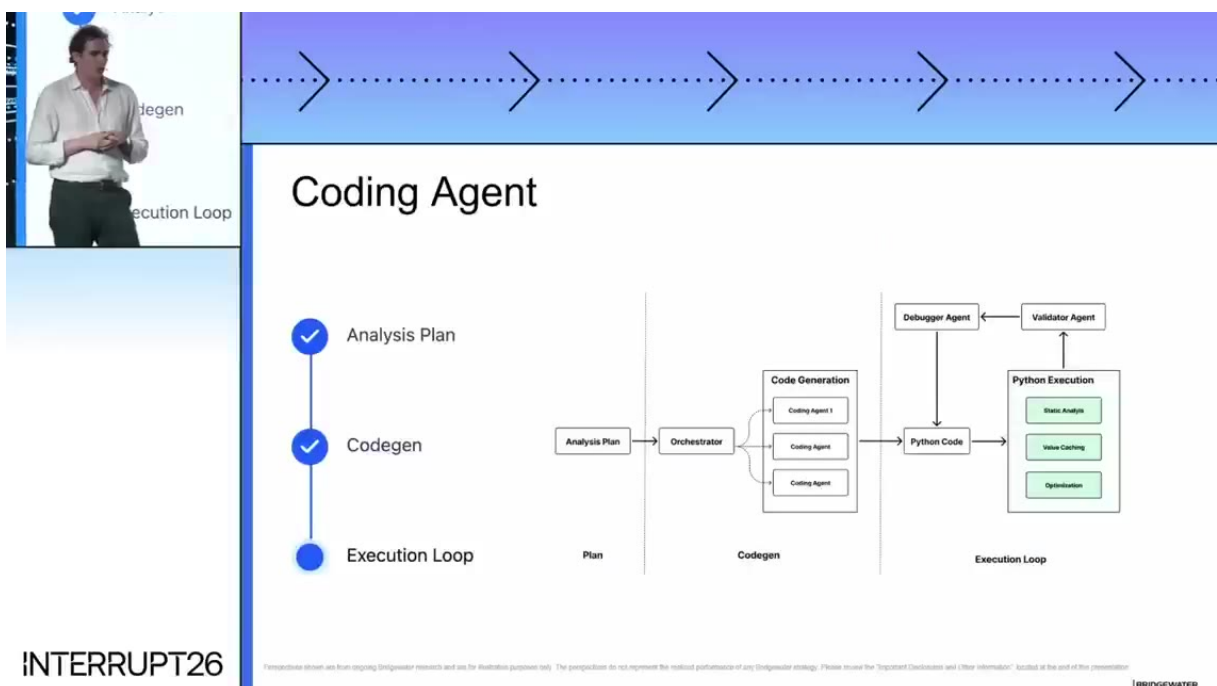
So the first thing is we split it into tasks, and then we do parallel LLM generation. And because our plan is so detailed, a visualization task at the end of the plan already knows everything it needs to consume code generation that hasn't yet been completed for loading data, say. And then when we compare this to Claude Code on our benchmark suite, you can see that in the mean, so this is for the same context and the same plan, we're about four times faster at generating code. But then we have this hyperscaling. So a 20-task plan takes the same amount of time as a 3-task plan.



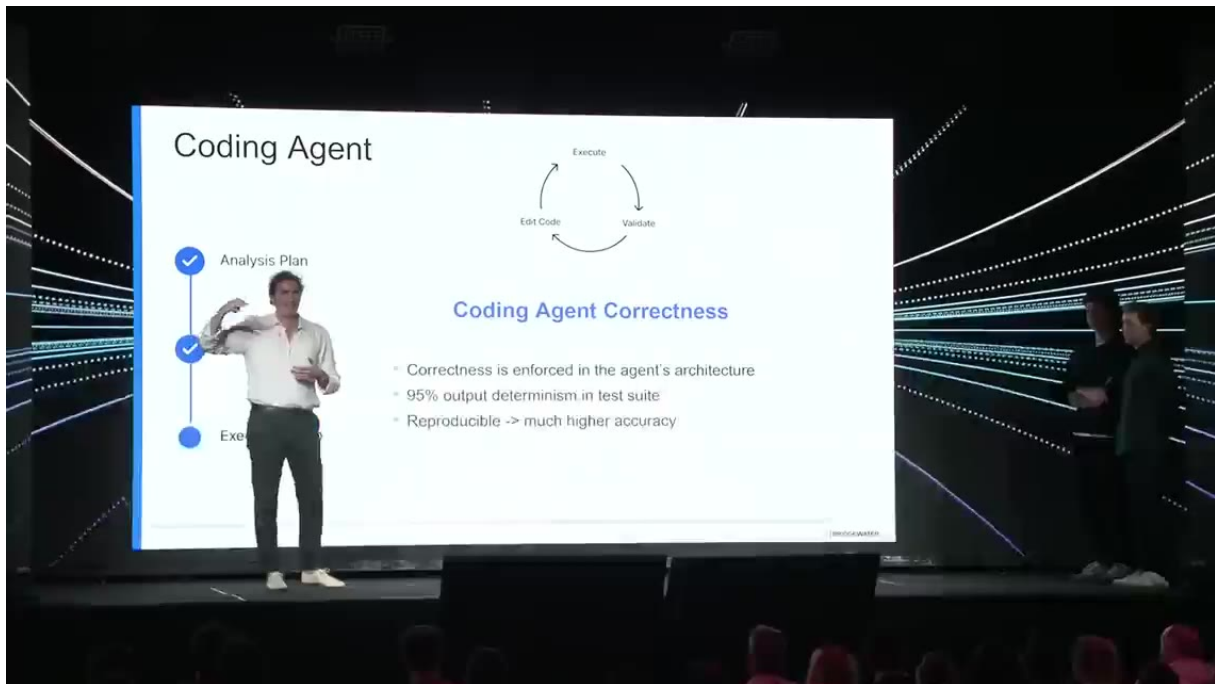
Benchmark — code generation for an analysis plan: Claude Code 67s vs. Pocket Analyst 17s

Okay, so now we have code and we're going to execute it. You would want to just execute the code naively, but sadly LLMs today aren't perfect on our tasks so they don't normally one-shot it. Instead what we do is we take the task that came in from the plan and then we have code that was generated from that task, and then we're going to run the code, compare it to the task, see if it's correct. And if it's not, then we'll edit the code and keep going until we're complete.

The first thing we do on the code is we do static analysis, and then figure out the DAG, and we're going to apply our validation agents in parallel. So here there's two tasks that are going to be validated at the same time. So a five-task plan comes down to three layers, and then a 20-task plan might be four or five layers of validation.



And then the main point I want you to take away from this part of the slides is that we enforce correctness in the architecture. So again, no agentic orchestration. This is regular Python code, and so the handlers are really quick, and the agents cannot forget to validate. They are forced to validate. The result is that when we run our test suite, any one plan from our test suite, 95% of the time, the code that comes out is exactly the same for two different agents. So it's like a deterministic coding agent is the idea. And then because we have such a reproducible agent, the idea is that as we're scaling and hill climbing and evaling, that we have something much more dependable than a vibes-based or LLM-as-judge evals.



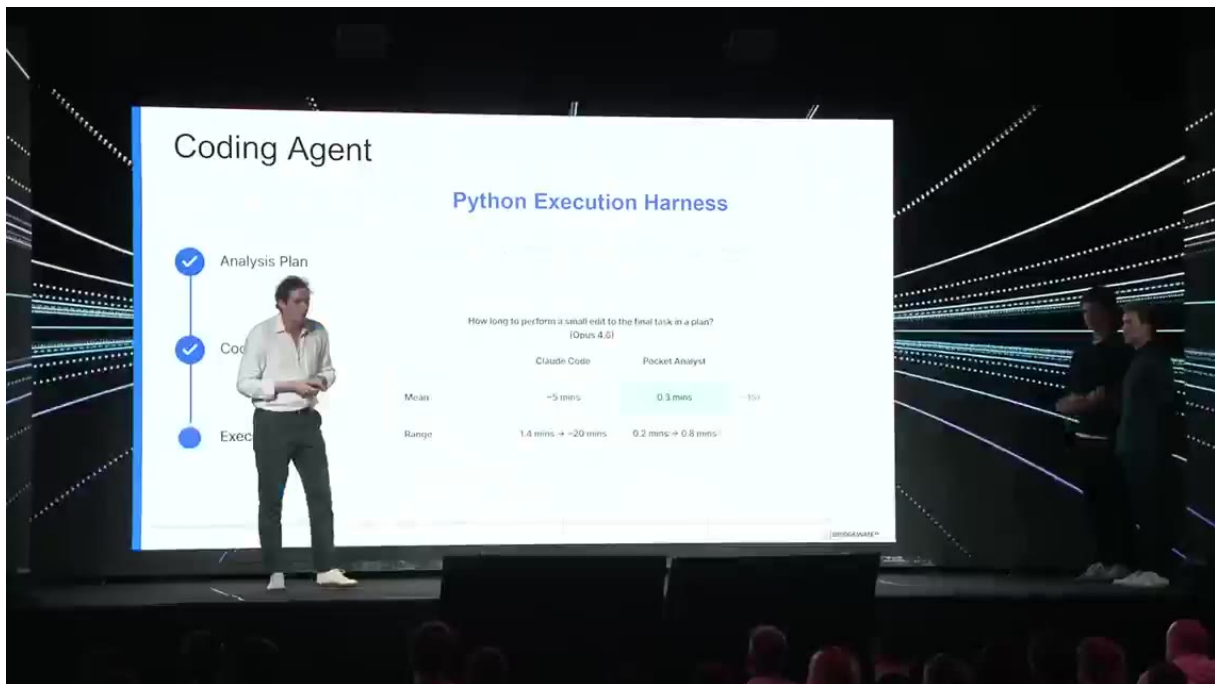
Coding Agent correctness — enforced in architecture, 95% output determinism

Cool. So there's one more topic I'm going to touch on here which is our execution layer. Normally coding agents, they invoke their code themselves. So they generate code and then they call it themselves via terminal. There's a couple of trade-offs here. That's higher latency with the tool calls, but I'm sure you've all experienced like they will sometimes get lost along the way. And so instead, we run the code for the LLMs.

We do a classical static analysis pipeline where we inject caching annotations into the Python code to avoid re-execution and run it through a custom framework. So this is like an example benchmark that we have for Claude invoking its own code versus Pocket Analyst. And you can see we're, again, faster because we're never double loading data or double executing intermediates. But the actual win is not on the first time you run the code, it's the second time.

So this is a benchmark where we take the last chart in a plan and then just change the name. And so Claude Code is going to go rerun all the code. So it's basically the same amount of time, though it is faster at editing the code. But Pocket Analyst basically has instantaneous code execution for the second round. And this means that when you're working on the product as an investor, you can make small tweaks to your investment analysis without having to do that overload, the overhead of

a regular iteration.



Benchmark — small edit to the final task: Claude Code ~5 mins vs. Pocket Analyst 0.3 mins

Okay, so we're out of time. We had many learnings, but unfortunately we can only stick to these. The first takeaway I think is that we really believe in specializing our agents. So we don't really believe in generic powerful agents. They make really cool demos, I'm sure many of us have given them. But it's really hard to make that a daily workflow that you can depend on. Instead, we take often very narrow workflows, and then benchmark them very heavily. And then we hill climb those benchmarks. And then you can compound the agents after the fact, but it's harder to go back the other way.

Pocket Analyst: Technical Takeaways

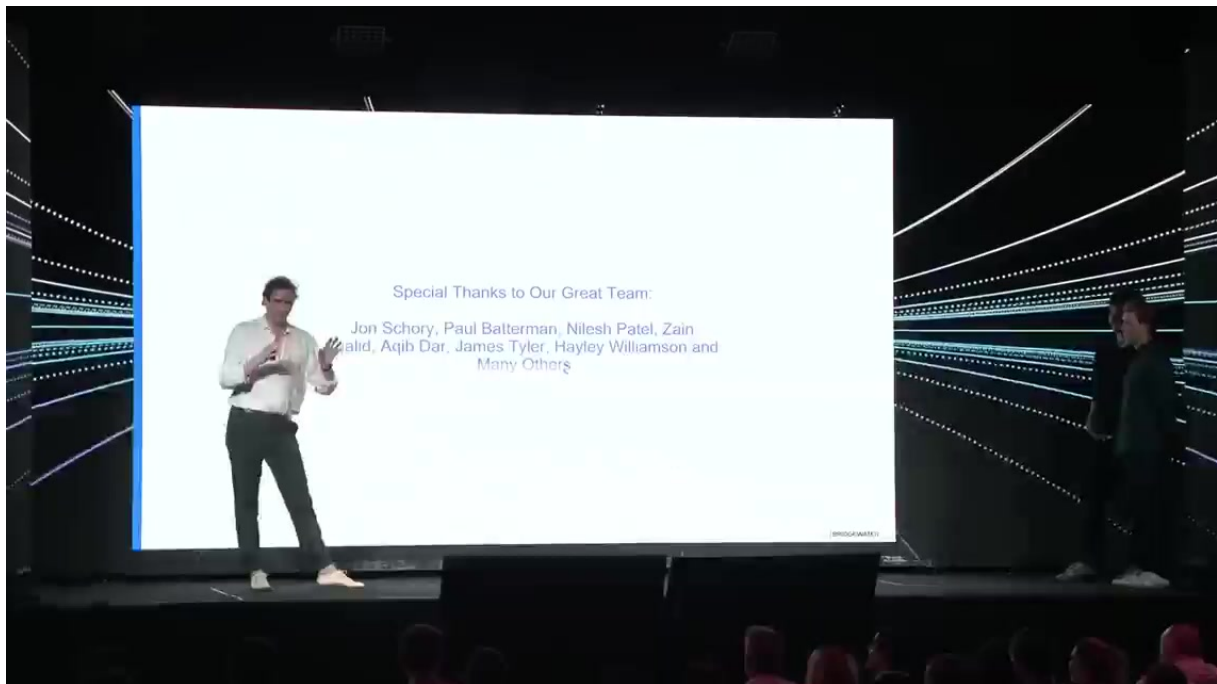
- Isolate your agents -> Specialization
- Automate *actual user workflows*
 - Avoid generic "powerful" agents!
 - Compound agents trained on specific tasks (e.g. sub-agents)

INTERRUPT26

Technical takeaways — isolate your agents, automate actual user workflows, avoid generic powerful agents

And this last thing is kind of like a thought exercise and hopefully it's exciting that someone might take away, is thinking of agentic coding as a compiler problem, not as an agentic problem. Compilers have already been generating code for decades, and they have a ton of techniques for how you can generate code more reliably, correctly, and deterministically. And so yeah, we'd love to chat about this.

With that, I'd like to shout out our direct team. There were many others, but this is the direct team. And, thank you. We're going to be outside at the AMA. We would really love to chat and talk about it and see if you are excited by the problems too. And then see you all around tonight. Thank you so much.



Special thanks — the direct team